

Lec-27

`get()` navigates to a URL.

`manage().window().maximize()` ensures the browser is maximized.

`manage().window().minimize()` minimizes the browser to save screen space.

```
ChromeDriver driver = new ChromeDriver( );
driver.manage( ).window( ).maximize( );
driver.get("https://tutorialsninja.com/demo/");
driver.manage( ).window( ).minimize( );
```

1 `get()` Method

Purpose:

- Used to **navigate to a specified URL** in the browser.

Usage:

```
driver.get("https://tutorialsninja.com/demo/");
```

Key Points:

- The **URL must include the protocol (e.g., `http://` or `https://`)**.
 - If you provide just `www.example.com` without `http://`, you will get an **`InvalidArgumentException`**.
- This method **waits until the page is fully loaded** before proceeding to the next step in the test.

Example:

```
WebDriver driver = new ChromeDriver();

driver.get("https://example.com");
```

2 `manage().window().maximize()`

Purpose:

- Maximizes the browser window to **full screen**.

Usage:

```
driver.manage().window().maximize();
```

Key Points:

- Ensures that the browser is fully visible during execution, which can help in avoiding issues with elements being out of view.
- It's typically used **immediately after launching the browser**.

Example:

```
WebDriver driver = new ChromeDriver();  
driver.manage().window().maximize();  
driver.get("https://example.com");
```

3 `manage().window().minimize()`

Purpose:

- Minimizes the browser window.

Usage:

```
driver.manage().window().minimize();
```

Key Points:

- This can be useful if you want to **free up screen space** during automated execution or if you want to run tests without showing the browser.

Lec-28

```
// Launch browser and navigate to URL
ChromeDriver driver = new ChromeDriver();
driver.manage().window().maximize();
driver.get("https://tutorialsninja.com/demo/");

// Find and interact with the search box
WebElement searchBoxField = driver.findElement(By.name("search"));
searchBoxField.click(); // Focus on the search box
searchBoxField.sendKeys("HP"); // Type "HP" in the search box

// Find and click the search button
WebElement searchButton =
driver.findElement(By.xpath("//button[contains(@class,'btn-default')]"));
searchButton.click(); // Submit the search

// Verify if the HP product link is displayed
WebElement hpProductLink = driver.findElement(By.linkText("HP LP3065"));
boolean productDisplayStatus = hpProductLink.isDisplayed();

// Validate the test result
if (productDisplayStatus) {
    System.out.println("Test Passed");
} else {
    System.out.println("Test Failed");
}
```

1 findElement()

Purpose:

- The `findElement()` method is used to locate a **single web element** on a web page.
- It returns an object of type `WebElement`.

Usage in the Code:

```
<input type="text" name="search" value="" placeholder="Search"
class="form-control input-lg">
```

```
WebElement searchBoxField = driver.findElement(By.name("search"));
```

- `By.name("search")`: Specifies that the element should be located by the `name` attribute with the value `"search"`.

2 WebElement

Purpose:

- Represents a web element (e.g., input field, button, link) in the DOM.
- Once located, the `WebElement` object provides methods to interact with the element.

Usage in the Code:

```
WebElement searchBoxField = driver.findElement(By.name("search"));
searchBoxField.click();
searchBoxField.sendKeys("HP");
```

- `searchBoxField`: Stores the located element (search box) so you can interact with it.
- `.click()`: Simulates a mouse click on the element.
- `.sendKeys("HP")`: Simulates typing the text `"HP"` into the search box.

3 By

Purpose:

- `By` is a **locator strategy class** that defines how to locate elements on a web page.

Common Locator Strategies:

- `By.name()`: Locates elements by the `name` attribute.

- **By.xpath()**: Locates elements using an XPath expression.
- **By.linkText()**: Locates links by their exact visible text.

Usage in the Code:

```
WebElement searchBoxField = driver.findElement(By.name("search"));
WebElement searchButton =
driver.findElement(By.xpath("//button[contains(@class,'btn-default')]"));
WebElement hpProductLink = driver.findElement(By.linkText("HP LP3065"));
```

```
<button type="button" class="btn btn-default btn-lg"><i
class="fa fa-search"></i></button>
```



- **By.name("search")**: Finds the search box using its `name` attribute.
 - **By.xpath("//button[contains(@class,'btn-default')]")**: Finds a button with a class containing `btn-default`.
 - **By.linkText("HP LP3065")**: Finds a hyperlink (`<a>` tag) with the exact text "HP LP3065".
-

4 click()

Purpose:

- Simulates a mouse click on a web element.

Usage in the Code:

```
searchBoxField.click();
searchButton.click();
```

- The code clicks on the search box (`searchBoxField`) and the search button (`searchButton`) to perform an action.
-

5 sendKeys()

Purpose:

- Simulates typing into an input field.

Usage in the Code:

```
searchBoxField.sendKeys("HP");
```

- Enters the text "HP" into the search box.

6 isDisplayed()

Purpose:

- Checks if a web element is visible on the web page.

Usage in the Code:

```
boolean productDisplayStatus = hpProductLink.isDisplayed();  
if (productDisplayStatus) {  
    System.out.println("Test Passed");  
} else {  
    System.out.println("Test Failed");  
}
```

- **hpProductLink.isDisplayed()**: Verifies whether the link for "HP LP3065" is visible.
- The test passes if the element is visible and fails otherwise.

Lec-29

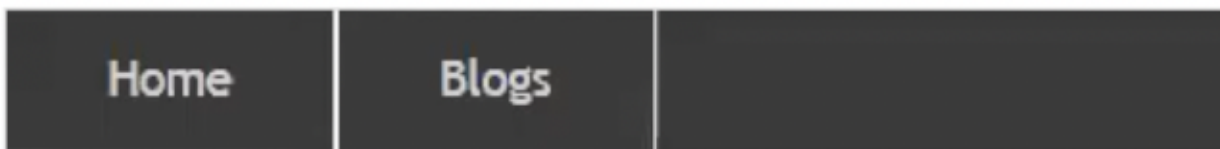
Summary Table of Actions and Elements

Action	Web Element	Method Used	Code Snippet
Open dropdown menu	My Account dropdown menu	<code>click()</code>	<code>myAccountDropMenu.click();</code>

Navigate to login	Login link	<code>click()</code>	<code>loginOption.click();</code>
Enter email	Email address field	<code>sendKeys()</code>	<code>emailAddressField.sendKeys("amoot oricap1@gmail.com");</code>
Enter password	Password field	<code>sendKeys()</code>	<code>passwordField.sendKeys("12345");</code>
Submit login	Login button	<code>click()</code>	<code>loginButton.click();</code>

Refer to this Link: <https://omayo.blogspot.com/>

Clicking Link



This blog is created for <http://www.Selenium143.blogspot.com>



```
WebElement selenium143Link = driver.findElement(By.id("link1"));
selenium143Link.click();
```

Clicking Checkbox options

By Default Selected Check Box option



```
WebElement blueCheckBoxField = driver.findElement(By.id("checkbox2"));
blueCheckBoxField.click();
```

To uncheck **Orange** and check **Blue**,

```
WebElement orangeCheckBoxField = driver.findElement(By.id("checkbox1"));
orangeCheckBoxField.click();

WebElement blueCheckBoxField = driver.findElement(By.id("checkbox2"));
blueCheckBoxField.click();
```

Clicking Radio option

```
WebElement maleOption = driver.findElement(By.id("radio1"));
maleOption.click();
```

Diff b/w TextBox & textArea Field

TextBox:

- Allows users to enter a **single line** of text.
- Used for short inputs like names, emails, or passwords.
- Example in HTML: `<input type="text">`

TextArea:

- Allows users to enter **multiple lines** of text.
- Used for longer inputs like comments, addresses, or messages.
- Example in HTML: `<textarea></textarea>`

The image shows a web form with two main components. On the left, there is a 'Text Area Field' with a label above it and a large rectangular input box below it. On the right, there is a 'Search This Blog' section with a small input box and a 'Search' button next to it.

Search This Blog here is **TextBox**

Filling text in TextBox:

```
WebElement textAreaField = driver.findElement(By.id("ta1"));
textAreaField.sendKeys("My name is Arun Motoori. My youtube channel name is:");
```

sendKeys() can be used with fields like **textbox**, **textarea**, and **password** to enter text.

The image shows a login form titled 'Returning Customer'. It has a sub-header 'I am a returning customer'. Below this, there are two input fields: 'E-Mail Address' containing the text 'asdfasdf' and 'Password' containing masked text '*****'. There is a link 'Forgotten Password' below the password field. At the bottom, there is a blue 'Login' button.

Text is toggled(* or .) in the password field.

To click on myAccount,

```
By.xpath("//span[text()='My Account']"))
```

```
WebElement myAccountDropDownMenu = driver.findElement(By.xpath("//span[text()='My Account']"));
myAccountDropDownMenu.click();
```

```
WebElement loginOption = driver.findElement(By.linkText("Login"));
loginOption.click();
```

```
WebElement emailAddressField = driver.findElement(By.id("input-email"));
emailAddressField.sendKeys("amotooricap1@gmail.com");
```

```
WebElement passwordField = driver.findElement(By.id("input-password"));
passwordField.sendKeys("12345");
```

WebElement loginButton =

```
driver.findElement(By.xpath("//input[@value='Login']"));
```

loginButton.click();

If successfully logged in, we should be able to see link with text “Edit your account information”

WebElement

editYourAccountInformationOption =

```
driver.findElement(By.linkText("Edit your account information"));
```

```
boolean displayStatus = editYourAccountInformationOption.isDisplayed();  
if(displayStatus) {  
    System.out.println("User got logged in successefully. Test Passed");  
} else {  
    System.out.println("User login failed. Test Failed");  
}
```